NHAI EdgeLock

Enterprise-Grade Offline-First Face Authentication
& 3D Liveness Detection

Hackathon Project – Binary Brains

Complete Technical Guide
Codebase Walkthrough · GhostFaceNet Deep Dive · Demo FAQ

Prepared for NHAI Hackathon 7.0

Table of Contents

1 Project Overview

NHAI EdgeLock is a smartphone-based face authentication system designed for the National Highways Authority of India (NHAI). It enables field workers (toll operators, maintenance crews, patrol officers) to verify their identity using only the phone's camera – **completely offline, with no internet required.**

The system combines three core capabilities:

- **3D Liveness Detection** – Prevents photo/video spoofing with blink, head-turn, and depth checks
- **Face Recognition** – GhostFaceNet AI model (~1 MB) converts faces into 512-number fingerprints
- **Offline-First Audit Trail** – Every authentication is logged with GPS + timestamp and synced to cloud when available

Key Facts

Metric	Value
Model Size	~1 MB (GhostFaceNet INT8 quantized)
Recognition Speed	~10-50 ms per inference
Total Auth Time	~3-5 seconds
Database	SQLite (encrypted embeddings + logs)
Encryption	AES-256, hardware-backed keys
Anti-Spoofing	Blink + Head Turn + 3D Depth
Storage per User	~2 KB per face fingerprint

2 Architecture Pipeline

The authentication pipeline follows this data flow:

Camera Feed → MediaPipe Face Mesh (Liveness WebView) → JPEG Decode (jpeg-js) → Face Alignment (CLAHE) → GhostFaceNet INT8 Embedding → Cosine Similarity Matching (SQLite) → Encrypted Local Logging → AWS Sync Queue

Step-by-Step Auth Flow

Step 1 – App Startup: Load SQLite database → Load GhostFaceNet TFLite model → Cache MediaPipe assets

Step 2 – Scan: User taps 'Verify' → Camera captures preview frame (640×480)

Step 3 – Liveness: Blink: MediaPipe detects 468 face landmarks → EAR (Eye Aspect Ratio) checked → User must blink

Step 4 – Liveness: Head Turn: Yaw angle calculated from nose-to-cheek asymmetry → User turns head

Step 5 – Depth Check: Standard deviation of z-coordinates checked → Photos/videos fail (flat z≈0)

Step 6 – Matching: 3 photos taken (180ms apart) → Averaged → GhostFaceNet produces 512-dim fingerprint

Step 7 – Database Comparison: Cosine similarity against all enrolled faces → Threshold check (0.75 single, 0.84 multi)

Step 8 – Logging: GPS captured → AuthLog saved to SQLite → Background sync to AWS endpoint

3 Complete Codebase Walkthrough

3.1 Root Files

- **package.json** – Project manifest with all dependencies (Expo, TFLite, SQLite, MediaPipe, etc.)
- **app.json** – Expo configuration: app name, icons, permissions (camera, location)
- **tsconfig.json** – TypeScript strict mode configuration
- **babel.config.js** – Babel/Expo transpiler setup
- **metro.config.js** – Metro bundler config (adds .tflite, .wasm asset extensions)
- **eas.json** – EAS Build profiles for iOS/Android cloud builds
- **jest.config.js** – Jest testing configuration
- **App.tsx** (root) – Simply imports and re-exports src/App.tsx

3.2 src/App.tsx – The Entry Point

This is the main control room. On launch it runs 3 sequential steps:

1. **Initialize SQLite** – Creates *enrolled_faces* and *auth_logs* tables
2. **Load TFLite Model** – Initializes GhostFaceNet recognition model
3. **Cache MediaPipe Assets** – Copies WASM/JS face detection files to device storage

It also mounts an **invisible WebView** (0×0 pixels) that runs the MediaPipe FaceMesh engine in the background. Communication between RN and WebView happens via *injectJavaScript* and *onMessage* callbacks.

3.3 Types & Constants

src/types/index.ts – Core data contracts:

- **AuthLog** – log_id, user_id, timestamp, gps_lat/lng, device_id, similarity_score, photo_thumb
- **LivenessError** – code (TIMEOUT | SPOOF_DETECTED | NO_FACE_DETECTED) + message
- **FaceAuthenticatorProps** – Component props with callbacks and configuration

src/constants/config.ts – Tunable thresholds:

- `SIMILARITY_THRESHOLD = 0.84 (multi-user match threshold)`
- `SIMILARITY_SINGLE_USER_THRESHOLD = 0.75 (single-user, more lenient)`
- `SIMILARITY_HIGH_CONFIDENCE = 0.91 (fast-path, bypasses margin checks)`
- `MIN_MATCH_MARGIN = 0.05 (winner must beat runner-up)`
- `MIN_PREPROCESS_VARIANCE = 0.05 (rejects blank/featureless images)`
- `LIVENESS_TIMEOUT_MS = 15000 (15s total for all challenges)`
- `REQUIRED_CHALLENGES = 2 (blink + head turn)`
- `AWS_SYNC_URL = '...' (cloud endpoint for log sync)`

3.4 Services – The Engine Room

services/database/sqlite.ts

Opens *binary_brains.db* via expo-sqlite. Provides *executeSql()* helper wrapping SQL transactions in Promises. Creates two tables on init:

- **enrolled_faces**: user_id (PK), embedding (BLOB, AES-256 encrypted), enrolled_at

- **auth_logs**: log_id (PK), user_id, timestamp, gps_lat/lng, device_id, similarity_score, photo_thumb, synced

services/database/enrolledFaces.ts

Stores face embeddings encrypted with AES-256. *insertEnrolledFace()* encodes Float32Array → base64 → AES encrypt → SQLite. *getAllEnrolledFaces()* reverses the process.

services/database/authLogs.ts

Inserts auth logs, retrieves unsynced logs (*synced=0*), and deletes server-confirmed logs (partial-batch safe by log_id list).

services/encryption/secureStorage.ts

Uses *react-native-encrypted-storage* (iOS Keychain / Android Keystore) to store a 256-bit AES key. *encryptData()/decryptData()* wrap CryptoJS.AES with the cached key. The key persists across app restarts in hardware-backed secure storage.

services/location/geolocation.ts

Requests foreground location permission and retrieves GPS coordinates with Balance accuracy (~1s budget). Returns null if permission denied or lookup fails.

services/network/awsSync.ts

Offline-first sync engine. *syncAuthLogs()* checks connectivity via NetInfo, fetches unsynced logs, POSTs them as JSON to the AWS endpoint (15s timeout), and only deletes logs that the server confirms in *received_logs*. *triggerSyncOnConnect()* subscribes to NetInfo events for auto-sync on reconnect.

3.5 AI & Recognition

services/ai/liveness.ts – LivenessEngine

A state machine managing facial liveness detection:

- **READY** – Waiting for face – transitions to first challenge
- **WAITING_BLINK** – Calculating EAR (Eye Aspect Ratio) – threshold < 0.33
- **WAITING_HEAD_TURN** – Calculating smoothed yaw angle – threshold > 0.12
- **PASSED** – All challenges completed successfully
- **FAILED** – Timeout (10s per challenge) or spoof detected

Uses a **rolling average** (window=5) for yaw smoothing and **hysteresis thresholds** (ON=0.12, OFF=0.09) to prevent flicker. The **depth consistency check** calculates standard deviation of z-coordinates across 468 landmarks – real 3D faces pass (std dev > 0.001), flat photos fail.

services/ai/mediapipeLandmarks.ts

Manages the hidden WebView running MediaPipe FaceMesh. Key operations:

- **ensureMediaPipeAssets()** – Copies 8 files (face_mesh.js, WASM binaries, model data) from app bundle to device local storage
- **writeMediaPipeHTML()** – Generates index.html with inline XHR polyfill (critical for Android file:// fetch bypass)
- **processImageForLandmarks()** – Sends base64 JPEG to WebView, receives 468 landmarks via postMessage
- **Debounce mechanism** – Drops frames while WebView is busy to prevent queue buildup

services/ai/recognition.ts – GhostFaceNet Integration

Core recognition service. *initRecognitionModel()* loads the 1MB INT8-quantized TFLite model via *react-native-fast-tflite*. *extractEmbedding()* performs:

1. Input quantization (float → int8 with scale=0.0078125, zero_point=-1)
2. Synchronous inference via *model.runSync()* (bypasses RN bridge via JSI)
3. Output dequantization (int8 → float with scale=0.14127, zero_point=24)
4. L2 normalization to unit length

findBestMatch() compares against enrolled faces using cosine similarity with multi-user logic (margin + ratio checks). *averageEmbeddings()* averages multiple captures index-by-index for stability.

services/ai/faceAlignment.ts

Performs similarity transform (rotation + scale + translation) using 5 canonical face points (eyes, nose, mouth corners) to warp detected faces to a standard 112×112 position. Falls back to center-crop resize if landmarks unavailable.

3.6 Camera & Image Processing

services/camera/frameProcessors.ts

processRecognitionFrame(): decodes JPEG → RGBA using jpeg-js → applies face alignment → applies adaptive contrast enhancement (CLAHE for extreme lighting, global histogram equalization for normal) → normalizes pixels to [-1.0, 1.0].

processLivenessFrame(): faster path (no CLAHE) that nearest-neighbor resizes to 320×240 for real-time liveness analysis.

utils/imagePreProc.ts

CLAHE implementation (8×8 tiles, bilinear interpolation, contrast limiting) for outdoor lighting normalization. Also includes global histogram equalization, Laplacian blur detection, brightness calculation, and frame quality gates (confidence ≥ 0.85, brightness 30-240, blur ≥ 50).

3.7 Database & Encryption

The encryption flow for enrolled faces:

- **1.** Generate 256-bit AES key → store in iOS Keychain / Android Keystore
- **2.** Convert Float32Array face embedding → base64 string
- **3.** Encrypt with AES-256 (CBC mode, CryptoJS)
- **4.** Store encrypted ciphertext in SQLite BLOB column
- **5.** On retrieval: decrypt → base64 decode → Float32Array

3.8 Network & Cloud Sync

The sync system is built with **zero-loss guarantees**. Logs are POSTed in batches to 'https://binary-brains-mock-aws.onrender.com/api/sync'. The server must echo back received_log_ids inside HTTP 200. Only those IDs are deleted from local SQLite. NetInfo listeners trigger auto-sync on connectivity restoration.

3.9 Hooks & Components

hooks/useAuth.ts

The main orchestrator hook managing the entire auth pipeline. Transitions: **idle** → **scanning** → **liveness** → **matching** → **authenticated/failed**. Handles 3-shot averaged capture, GPS retrieval, log insertion, and background sync trigger. Includes borderline retry logic: if a single-user match is within 0.03 of threshold, it retries once.

components/CameraOverlay.tsx

Displays camera feed with a semi-transparent overlay and animated pulsing face-shaped cutout. Border color changes: blue (idle/scanning), green (authenticated), red (failed).

components/FaceAuthenticator.tsx

Main UI component. Connects useAuth hook to CameraOverlay. Manages haptic feedback (light buzz on challenge change, heavy buzz on success, error buzz on failure). Shows status pill at bottom ('Hold still…', 'Please blink', etc.) and retry button on failure.

components/LivenessFeedback.tsx

Colored notification banners: orange (warning/instruction), green (success, auto-dismiss 1.5s), red (error with tap-to-dismiss).

3.10 Screens

screens/DemoAuthScreen.tsx

Main screen with: header (title + Enroll button), sync status badge (Synced/Pending/Offline), camera container with FaceAuthenticator, active auth result card, Verify button, and recent logs list (last 5 auth attempts from SQLite).

screens/EnrollmentScreen.tsx

Registration screen. Captures 3 face frames, checks quality (confidence/brightness/blur), extracts embeddings, averages them, encrypts, and stores in database against a user ID. Includes skip-quality option for testing.

4 GhostFaceNet – Why This Model?

4.1 Model Comparison

Model	Size	Speed	Accuracy	Best For
FaceNet	~90 MB	Slow	High	Servers
ArcFace	~200+ MB	Very Slow	Highest	Research
MobileFaceNet	~4-5 MB	Fast	Good	Mobile Apps
GhostFaceNet ★	~1 MB	Fastest	Good	On-Device Mobile

4.2 Four Reasons GhostFaceNet Was Chosen

1. Size – Only 1 MB (99% smaller than alternatives)

INT8 quantization compresses the model 4× vs float32. The app already bundles MediaPipe WASM files, so every MB counts – especially on budget Android phones common in highway field operations.

2. Speed – Synchronous ~10-50ms per inference

Uses *react-native-fast-tflite* with JSI (bypasses the React Native bridge for C++ direct calls). This enables the 3-shot averaging strategy (~600ms total) without noticeable delay. FaceNet inference would take 500ms+ per frame.

3. 512-Dimensional Embeddings – Just Right

512 numbers per face fingerprint. Stored as AES-256 encrypted BLOBs in SQLite (~2KB per user). Fast enough for linear scan matching across thousands of users.

4. ArcFace Loss Training – Hidden Superpower

GhostFaceNet is trained with ArcFace loss (same as the 200MB ArcFace model) but uses Ghost Modules to achieve comparable accuracy with 200× fewer parameters. Ghost Modules generate more features from fewer parameters by cheaply transforming a small set of intrinsic feature maps.

4.3 Recognition Pipeline Code Flow

```
1. Camera captures JPEG (640×480, quality=0.25) 2. jpeg-js decodes to RGBA pixel array 3. fastResize112x112: nearest-neighbor downsample to 112×112 RGB 4. INT8 quantization: floatVal / 0.0078125 + (-1) → Int8Array(37632) 5. model.runSync([int8Input]) → 512-element Int8Array output 6. Dequantization: (int8Val - 24) × 0.14127 → Float32Array(512) 7. L2 normalize to unit length 8. Cosine similarity against all enrolled embeddings 9. Threshold + margin + ratio check → Match or Reject
```

5 Demo FAQ – 33 Questions & Answers

Q1: What problem does this app solve?

NHAI needs to verify field worker identities at remote highway locations without internet. Our solution: smartphone-based face authentication working completely offline, with liveness detection and GPS audit trail.

Q2: Who is the target user?

Two types: (1) Field workers (toll operators, patrol, maintenance) who authenticate their identity. (2) Administrators who enroll new workers by capturing their face and assigning a user ID.

Q3: What makes this different from phone face unlock?

Four differences: (a) Multi-user support – unlimited workers, not just phone owner. (b) Liveness detection – blink + head turn + 3D depth, not just a photo match. (c) Audit trail – GPS + timestamp + photo every authentication. (d) Offline-first – no cloud dependency.

Q4: What is the tech stack?

Expo SDK 50 + React Native 0.73.6 + TypeScript (strict). MediaPipe FaceMesh for face detection (hidden WebView). GhostFaceNet INT8 TFLite model (~1MB) for recognition. expo-sqlite for local storage. AES-256 encryption via react-native-encrypted-storage. CLAHE for image contrast enhancement. REST API for cloud sync.

Q5: How does the app work offline?

Every component works offline: AI model runs on-device via TFLite (no cloud API calls). SQLite stores everything locally. Encryption keys in hardware-backed secure storage. Auth logs queue locally until internet is available, then auto-sync. Zero data loss: logs only deleted after server confirmation.

Q6: Walk us through the authentication pipeline step by step.

1) App starts → loads DB + model + assets (~3-5s). 2) User taps Verify. 3) Camera turns on (front, 640x480). 4) Scanning – 'Hold still…'. 5) Liveness – blink + head turn + depth check. 6) Matching – 3 photos → average → 512-dim fingerprint → database comparison. 7) Authenticated ■ or Failed ■. 8) GPS + log saved. 9) Background cloud sync.

Q7: Which AI model do you use and why?

GhostFaceNet (~1MB, INT8 quantized). Chosen because: FaceNet is 90MB, ArcFace is 200MB+ – too large for budget phones. MobileFaceNet is 4-5MB but less accurate. GhostFaceNet achieves best accuracy/size ratio. Runs synchronously in ~10-50ms via fast TFLite.

Q8: How do you prevent photo/video spoofing?

Three layers: (1) Active – blink detection (EAR < 0.33) and head turn (yaw > 0.12). (2) Passive – 3D depth check: real faces have varying z-coordinates (std dev > 0.001), flat photos have z≈0. (3) Image variance: faces have high texture variance (>0.05), walls/ceilings are uniform and get rejected.

Q9: How accurate is the face matching?

Thresholds tuned to model output: single user 0.75, multi-user 0.84 with 0.05 margin, high confidence fast-path at 0.91. Genuine accept rate ~95%+ with proper lighting. 3-shot averaging improves accuracy ~15-20% over single shot.

Q10: How do you protect stored face data?

AES-256 encryption with keys stored in hardware-backed secure storage (iOS Keychain / Android Keystore – same tech as banking apps). Face embeddings converted to base64 → encrypted → BLOB in SQLite. Even extracting the database file yields unreadable ciphertext.

Q11: What happens if someone steals the phone?

Face data is AES-256 encrypted – unreadable without key. Key is in hardware Keychain/Keystore, not extractable even with root. No cloud credentials stored on device. Device ID is install-specific, cannot impersonate on another device.

Q12: How does cloud syncing work?

Logs inserted with synced=0. When online: POST batch to AWS endpoint (15s timeout). Server responds with received_log_ids. Only those IDs are deleted locally. Auto-sync triggers on network reconnect via NetInfo listener. Manual 'Sync Now' button available.

Q13: What if there's no internet for days?

All authentication keeps working. Logs queue in SQLite indefinitely. Auto-sync triggers when connectivity returns. Visual sync badge shows status. Manual sync button for convenience.

Q14: Why use a hidden WebView for face detection?

MediaPipe FaceMesh is web-based (JS + WASM). A hidden WebView runs it cross-platform with the same code. Key challenge: Android blocks fetch() from file:// origins – solved with an XHR polyfill replacing fetch with XMLHttpRequest.

Q15: Why CLAHE for image processing?

Highway lighting is extreme: half face in sun, half in shadow. Simple histogram equalization works globally and can't handle local contrast issues. CLAHE divides the face into 8×8 tiles, enhances each independently, clips noise, and blends smoothly – making faces consistent regardless of outdoor lighting.

Q16: Why 3-shot averaging?

Micro-movements (blinks, sways, expression changes) between frames create variation. 3 captures 180ms apart, averaged, cancel out temporary noise. Gives ~15-20% better matching accuracy at the cost of ~2 seconds extra processing time.

Q17: How do you calculate if someone blinked?

Eye Aspect Ratio (EAR): (|p2-p6| + |p3-p5|) / (2×|p1-p4|) using 6 landmarks per eye. Normal eye ~0.35, closed eye < 0.33 threshold. 468 MediaPipe landmarks provide the coordinates.

Q18: How do you measure head turning?

Nose-to-cheek asymmetry: yaw = (dLeft - dRight) / (dLeft + dRight). Rolling average over 5 frames for smoothing. Hysteresis thresholds (ON=0.12, OFF=0.09) prevent flickering. Positive = right turn, negative = left turn.

Q19: How does the app handle poor lighting at night?

CLAHE enhances dark areas. Adaptive path: mean brightness < 30 or > 225 triggers full CLAHE; otherwise faster global equalization. Single-user threshold of 0.75 provides tolerance for lower-quality captures in darkness.

Q20: What happens if the user doesn't blink or turn?

10-second timeout per challenge. If no blink in 10s → FAILED with 'Timeout'. If 30 consecutive frames (~3s) with no face detected → 'Face lost'. User sees red error banner with Retry button.

Q21: What if someone looks like the enrolled user (impostor)?

Multi-user mode: three passes needed – score ≥ 0.84, margin ≥ 0.05 over runner-up, ratio ≥ 1.08 vs runner-up. Single-user: score ≥ 0.75. If ambiguous, returns 'Ambiguous match' rejection rather than false accept.

Q22: How do you handle glasses, masks, or helmets?

Glasses work fine (MediaPipe handles them). Masks: partial face still works, depth check still passes for real 3D face. Full-face helmets: 'No face detected'. Sunglasses may interfere with blink detection – head turn challenge still works.

Q23: What if the camera is damaged or dirty?

Quality gates during enrollment detect blur (Laplacian variance < 50) or bad lighting (brightness < 30 or > 240). During auth, 3 consecutive poor frames trigger helpful error.

Q24: How fast is authentication?

App startup: 3-5s. Face capture: ~0.3s. Liveness check: 1-3s. 3-shot capture + averaging: ~0.6s. TFLite inference: 10-50ms. Database lookup: ~1ms. Total: ~3-5 seconds on mid-range Android.

Q25: How many users can this support?

Each user ~2KB storage. SQLite handles millions of rows. Linear scan matching: 100 users in ~1ms, 1000 in ~10ms. Practical limit: thousands of users per device.

Q26: Can you show a spoof attack being detected?

Hold another phone showing a photo to camera. The app shows: 'Spoof detected: 3D depth consistency check failed.' in red. The photo has all z-coordinates ≈ 0 (flat), immediately flagged.

Q27: What features would you add next?

Voice challenge (say random number). Web dashboard with maps/analytics. Multiple model sizes (1-5MB). Federated enrollment across devices. Auto-expiry policies for re-enrollment.

Q28: What were the biggest technical challenges?

1) WebView file access on Android – fetch blocked from file://, fixed with XHR polyfill. 2) Highway lighting – solved with CLAHE. 3) Liveness jitter – solved with rolling avg + hysteresis. 4) EXIF rotation mismatch – fixed with skipProcessing=false. 5) Smile detection unreliable – replaced with head turn challenge.

Q29: How is this different from Face ID / Android Face Unlock?

Phone unlock: single owner only, no audit trail, varying spoof protection. Our system: unlimited workers, GPS + timestamp logging, blink + head turn + 3D depth anti-spoofing, cross-platform, adjustable thresholds (0.75-0.91).

Q30: Where would you deploy this in production?

Toll plazas (shift start verification), highway patrol (checkpoint check-ins), maintenance depots (access control), construction sites (worker verification), remote supervision (GPS location confirmation).

Q31: What makes this project innovative?

Three innovations: (1) Offline-first face recognition with 1MB model – no cloud needed. (2) Multi-layered anti-spoofing on standard phone camera (no IR sensor). (3) CLAHE from medical imaging adapted for highway lighting conditions.

Q32: How was this validated?

9 Jest test files (unit + integration + E2E). Mock AWS server for sync testing. 3-shot averaging validated vs single-shot (15-20% improvement). Thresholds tuned based on field logs (genuine scores 0.73-0.92).

Q33: What's the most clever piece of code?

The XHR polyfill for the hidden WebView – a 15-line fix essential for cross-platform operation. Android blocks fetch() from file:// origins but allows XHR. This polyfill intercepts MediaPipe's internal fetch() calls and routes them through XMLHttpRequest, making the same code work on both iOS and Android without platform-specific branches.

Generated from the NHAI EdgeLock codebase analysis. For questions, refer to the project README or source code.